Simulation of Iris Quadrotor Path Planing System Using RRT Algoritm

Maharani Yassar Dewanti[1]
Robotics and Artificial Intelegence Engineering
Airlangga University
Surabaya, Indonesia
[1]maharani.yassar.dewanti-2021@ftmm.unair.ac.id

***Abstract*—This research presents a detailed simulation study of the Iris quadrotor's path planning system using the Rapidly-Exploring Random Tree (RRT) algorithm. With the aim of enhancing the efficiency and safety of autonomous aerial vehicles, particularly quadrotors, in dynamic environments, this study leverages the PX4 platform as the foundational drone platform and integrates it with ROS for communication and visualization. The RRT algorithm, known for its ability to rapidly explore feasible paths in high-dimensional spaces, is implemented to generate optimal trajectories for the quadrotor's navigation. The simulation framework provides a controlled environment for testing and validating the path planning system, offering insights into its performance and potential real-world applicability. By simulating various scenarios, including obstacle avoidance and waypoint navigation, the study evaluates the effectiveness of the RRT algorithm within the PX4 framework. Through rigorous testing and analysis, the study demonstrates the capability of the RRT algorithm to generate efficient paths while ensuring feasibility and safety. Furthermore, the study explores optimization techniques to streamline the generated paths, aiming to minimize the number of waypoints required for navigation. By optimizing the paths, the study enhances the efficiency of the quadrotor's navigation process without compromising on safety or feasibility. Statistical analysis of path cost and time provides quantitative insights into the performance of the optimized paths, highlighting their effectiveness in real-world scenarios. Overall, the simulation study showcases the potential of the Iris quadrotor's path planning system, powered by the RRT algorithm within the PX4 framework, to navigate complex environments autonomously. The findings contribute to the advancement of autonomous aerial vehicle technology and pave the way for practical applications in surveillance, mapping, and delivery systems. (*Abstract*)**

Keywords—component, formatting, style, styling, insert (key words**)**

I. Introduction (*Heading 1*)

The burgeoning field of autonomous aerial vehicles has seen significant advancements, particularly in the development of quadrotors for various applications such as surveillance, mapping, and delivery systems [1,2,3]. A critical aspect of these systems is effective path planning to ensure safe and efficient navigation in dynamic environments [4].

Common path-planning algorithms can be classified as follows. Graph-based planning methods include A* [9], D* [10], and their optimization algorithms [11,12]. Since this type of algorithm is not suitable for path searching in high-dimensional space, it is often used for unmanned vehicles for path planning in the two-dimensional plane. Bionic-based planning methods, such as GA [13], ACO [14,15], and PSO [16], is a class of algorithms more often used to deal with patrols or secondary path optimization. Potential-field-based planning methods mainly include APF and its optimization algorithms [17,18,19], which have the problem of falling into local optimization. Sampling-based planning methods, such as RRT [20] and PRM [21], can be used to solve high-dimensional path-planning problems. In addition, their advantages are more obvious as the dimensionality increases. For this reason, the study of RRT and its optimization algorithm has received considerable attention from scholars in recent years.

Steven M. LaValle proposed the RRT algorithm [20] in 1998 to solve the robot path-planning problem, attempting to expand the path through node iteration, making the path branch like a tree until the branch touches the target point. Kuffner et al. proposed RRT-connect [22] in 2000, which introduces a dual-tree expansion strategy and adds a greedy strategy to the expansion of auxiliary trees to reduce the map complexity by expanding both trees simultaneously. Karaman et al. proposed the RRT* [23] algorithm in 2011, which improves path quality through root node reconstruction, sacrificing small time costs and saving a large amount of path costs, making RRT* the most successful variant of RRT. In 2019, Jeong proposed a Q-RRT* [24] algorithm based on RRT*, which optimizes the path cost of each node under the current expansion state through root node iteration. In addition, the development of RRT has also derived a series of optimization algorithms such as BG-RRT [25], Informed RRT* [26], PQ-RRT* [27], F-RRT* [28], and DPRRT* [29].

This paper presents a comprehensive simulation study of the Iris quadrotor's path planning system utilizing the RRT algorithm, with PX4 serving as the foundational drone platform. PX4, an open-source flight control software, offers extensive capabilities and flexibility for developing sophisticated autonomous flight applications. By leveraging PX4's powerful features and the efficiency of the RRT algorithm, this study aims to demonstrate an effective approach to quadrotor path planning in simulated environments. The simulation environment provides a controlled setting to test and validate the proposed path planning system, offering insights into its performance and potential real-world applicability. Through a series of simulations, we evaluate the Iris quadrotor's ability to navigate through predefined waypoints and avoid obstacles, highlighting the strengths and limitations of the RRT algorithm within the PX4 framework.

A. Problem Definition

If $X = (0,1)^n$ is an n-dimensional constructed space. This space consists of two main regions. X_{hinder} is the obstacle region, which contains obstacles or hindrances. X_{free} is the free region, which is free of obstacles and traversable. The conditions that must be satisfied are;

$$X_{free} \cap X_{hinder} = \emptyset \qquad (1)$$

$$X_{free} \cup X_{hinder} = X \qquad (2)$$

$$X_{init} \in X_{free} \qquad (3)$$

$$X_{goal} \in X_{free} \qquad (4)$$

The first equation shows there is no overlap between the free region and the obstacle region. And for the second equation shows there is an union of the free region and the obstacle region covers the entire space X. A path-planning problem is defined by $(X_{free}, X_{init}, X_{goal})$. where (3) is the starting point and (4) is the goal point. The objective is to find a continuous path Φ from X_{init} to X_{goal} that remains entirely within the free region. Mathematically, this means (5) is a continuous function with bounded variation such that for every τ in the interval (6).

$$\Phi : [0,1] \rightarrow X \qquad (5)$$

$$\tau \in [0,1], \Phi(\tau) \in X_{free} \qquad (6)$$

$$\Phi(0) = X_{init} \qquad (7)$$

$$\Phi(1) \in X_{goal} \qquad (8)$$

$$c(\Phi) = \min\{c(\Phi) \mid \in X_{free}\} \qquad (9)$$

Effective path planning involves finding a feasible path Φ that meets these criteria, specifically ensuring that the path starts at the initial point (7) and ends in the goal region (8), while avoiding all obstacles. Optimal path planning goes a step further by not only finding a feasible path but also minimizing the path cost $c(\Phi)$. The optimal Φ path is defined as the path with the lowest possible cost from X_{init} to X_{goal} while still satisfying the conditions for an effective path. Thus, the path cost $c(\Phi)$ is minimized such that in (9), ensuring both efficiency and safety in navigation through the n-dimensional space.

II. Simulation System And Methods

A. Configuration of Simullation Systems

The configuration of simulation system proposed in this study is based on ROS framework, as described on Fig. 2. The main components of the simulation system are divided into three parts, i.e. model of quadrotor system on Gazebo, PX4's SITL (Software In The Loop) and RRT path planning algorithm. To provide the flight controller for quadrotor model, SITL that runs PX4 firmware is utilized so that we can give commands from terminal linux for hexacopter model on Gazebo. SLAM algorithm is applied on ROS so that Gazebo and SITL can connected for subscribing or publishing data. As for the visualization of 3D map which is generated by SLAM, we used Rviz as the available GUI (Graphical User Interface) on ROS.

B. Iris's Quadrotor Model on Gazebo

The quadrotor UAV is usually assumed to be a rigid body with six degrees of freedom. The system state vector is defined as $X = [x\ y\ z\ u\ v\ w\ \varphi\ \theta\ \psi\ p\ q\ r]^T$, where x, y, z and u, v, w denotes position and velocity in the North-East-Down inertial frame Γ_I. φ, θ, ψ denotes Euler angles in roll, pitch, and yaw axes, respectively, and p, q, r denotes angular velocities in the body frame Γ_B, respectively. Figure 1 shows the quadrotor coordinate systems.

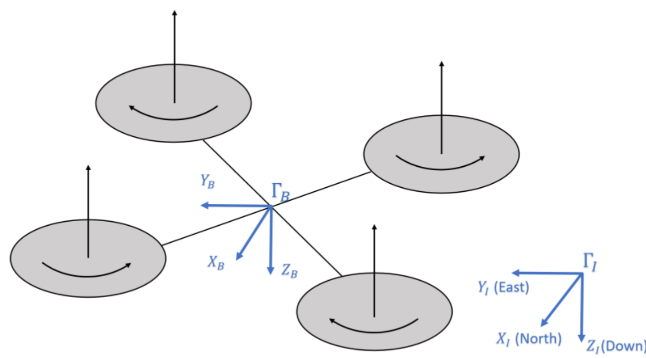

Figure 1. The quadrotor sketch with inertial frame Γ_I and body frame Γ_B.

The system is controlled by four control inputs $U = [U_1, U_2, U_3, U_4]$, where U_1 is the thrust force along the Z_B direction, and U_2, U_3, U_4 are rolling, pitching, and yawing moments in Γ_B, respectively. Quadrotor nonlinear dynamic model is constructed based on the Newton–Euler formalism;

$$\begin{cases} \dot{x} = u \\ \dot{y} = v \\ \dot{z} = w \\ \dot{u} = -(\cos\phi \sin\theta \cos\psi + \sin\phi \sin\psi)\dfrac{U_1}{m} \\ \dot{v} = -(\cos\phi \sin\theta \cos\psi - \sin\phi \cos\psi)\dfrac{U_1}{m} \\ \dot{w} = -(\cos\phi \cos\theta)\dfrac{U_1}{m} + g \\ \dot{\phi} = p + (\sin\phi \tan\theta)q + (\cos\phi \tan\theta)r \\ \dot{\theta} = q\cos\phi - r\sin\phi \\ \dot{\psi} = \dfrac{\sin\phi}{\cos\theta}q + \dfrac{\cos\phi}{\cos\theta}r \\ \dot{p} = \dfrac{(I_y - I_z)qr + U_2}{I_x} \\ \dot{q} = \dfrac{(I_z - I_x)pr + U_3}{I_y} \\ \dot{r} = \dfrac{(I_x - I_y)pq + U_4}{I_z} \end{cases} \tag{10}$$

Where m is the mass of the quadrotor, g is the gravitational acceleration, and IX, IY, IZ are quadrotor moment of inertia around three axes. The nonlinear model in (1) is widely used as the prediction model in the MPC scheme. Note that this model only takes gravitational force and control inputs into consideration

This study carry out software in the loop (SITL) simulations by the PX4 open-source flight control platform [23], using an Iris quadrotor model in the Gazebo robot simulator shown in Figure 2. The simulated quadrotor communicates with PX4 using MAVLink API, which defines a set of messages to supply sensor data from the simulated world to PX4 and return motor and actuator values applied to the simulated vehicle [24].

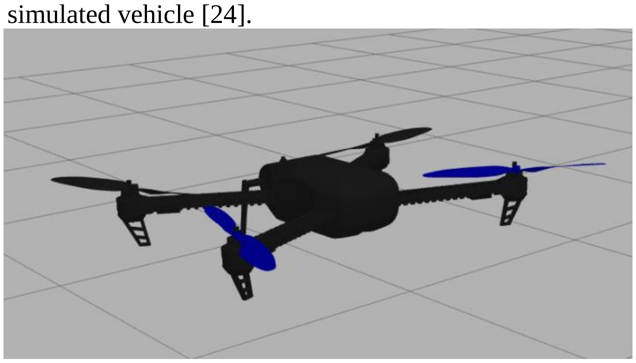

Figure 2. Iris quadrotor UAV model in Gazebo simulator.

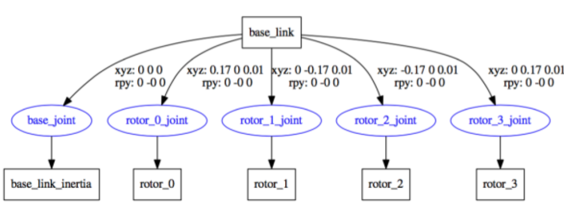

Figure 3. URDF of quadrotor model.

C. Robot Operating System (ROS)

ROS is a framework for incorporating various modules of robotics by configuring modules as a TCP network. The ROS community is currently growing in a very fast pace both in the community and industry. ROS can be used with PX4 and the Gazebo simulator. As the picture below indicates, PX4 communicates with Gazebo to receive sensor data from the simulated world and send motor and actuator values. It communicates with the GCS and ROS to send telemetry from the simulated environment and receive commands.

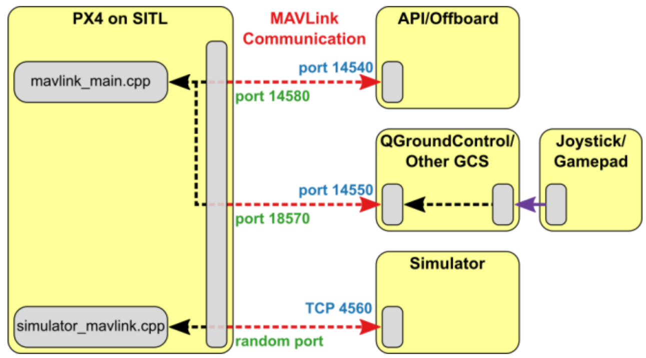

Figure 4. PX4's communication structure.

a. MAVROS

MAVROS is a ROS package which enables ROS to interact with autopilot software using the MAVLink protocol. MAVlink consists of 17 bytes and includes the message ID, target ID and data. The message ID shows what the data is. Message IDs can be seen in the messageID command set. This enables MAVLink to be able to get information from multiple UAVs if messages are transferred in the same channel. Messages can either be transmitted through wireless signals.

b. RQT

RQt is a collection of plugins that is part of ROS, a robot operating system. It provides a user-friendly way to interact with a robot and its sensors, including visualizing and analyzing sensor data, sending commands, and changing parameters. RQt also includes common plugins for debugging and visualization, such as a profiler, logger, and plotter, and allows users to create custom plugins. RQt consists of two metapackages: rqt for core infrastructure modules and rqt_common_plugins for debugging tools. This study used rqt for show nodes communication graph.

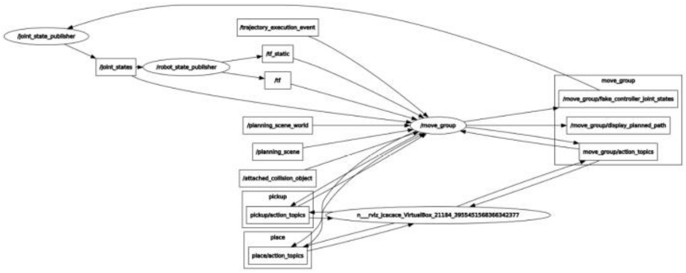

Figure 5. URDF Graph of communication between nodes using topics

c. RViz
RViz is a 3d visualization program for robots using ROS. RViz can be extended by writing plugins. librviz is a library for including rviz visualization capabilities. This study using Rviz for visualize the UAV's path iterations.

D. Rapidly-Exploring Random Tree

The RRT algorithm works by building a tree data structure from a root node located for instance where the vehicle is currently situated. Samples are drawn from the configuration space of the vehicle, and if it is a free sample (not inside an obstacle or in unknown space) it will be used to construct a new vertex in the graph. The next step is to search the tree to find the vertex which is nearest to the newly sampled point, using some distance metric. A new configuration is then found by moving an incremental distance from the nearest vertex towards the sampled configuration (steering). This assumes the movement is possible and not limited by some constraint, such as an obstacle for instance. If successful, the new configuration is added as a new node in the search graph. This procedure is repeated for a predetermined number of nodes or some time limit is reached. If a node is near the goal path be constructed by backtracking the vertices in the tree all the way back to the root vertex.

"Speed" is the biggest advantage of RRT, and randomness is also its biggest advantage, which makes it free from the trouble of falling into local optimization. The solution idea for the feasible path is to quickly expand the path like a tree, spreading from the starting point X_{init} to the surrounding areas until the target point X_{goal} is searched. It is very common to address the dynamic programming problem by constructing a weighted graph $G = (V,E)$, where $V = \{x_1, x_2, x_3, \cdots x_n\}$ is the vertex set composed of random tree nodes $x_i\ (i \in \{1,2,\cdots n\})$ and E is the edge set. Algorithm 1 provides a basic RRT construction idea, which is an iterative process that tries to generate a feasible new node $xnew$ for each iteration to expand the random tree after setting the end condition.

TABLE I. ALGORITHM OF RRT

Algorithm of RRT

Input : Map, X_{init}, X_{goal}

Output : G=(V,E);

: $V = \{X_{init}\}, E = \emptyset, i = 0$;

: While *True* do

: $x_{rand} \longleftarrow Sampling\ (i)$;

: $i \longleftarrow i + 1$;

: $x_{nearest} \longleftarrow Nearest\ (G, x_{rand})$;

: $x_{new} \longleftarrow GetNode\ (x_{rand},\ x_{nearest})$;

: if $ObstacleFree\ (x_{new},\ x_{nearest})$ then;

: $V \longleftarrow V \cup \{x_{new}\}$;

: $V \longleftarrow E \cup \{\ x_{nearest}, x_{new}\}$;

: if $x_{new}\ \approx\ x_{goal}$ then

: return (*V*, *E*);

The original procedure used in the RRT algorithm is described below:

a. $Sampling()$

Given a weighted map G=(V,E), it returns a random coordinate point in the map as a guide point $xrand$

b. $Nearest()$

Given G=(V,E) and guide point $xrand$, it calculates the point closest to $xrand$ in V as the extended proximal point $xnearest$, and returns it.

c. $GetNode()$

Given $xnearest$ and $xrand$, it takes $xnearest$ as the starting point $xrand$ as the guide, intercepts the point at which the set step length of the upper guide distance $xnearest$ is $xnew$, and returns to $xnew$.

d. $ObstacleFree()$

Given $xnearest$ and $xnew$, it returns $True$ or $False$ after collision checking. In addition, the calculated distances are all Euclidean distances.

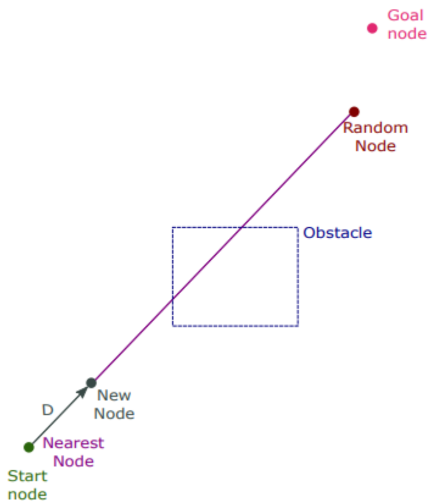

Figure 6. RRT First Iterate

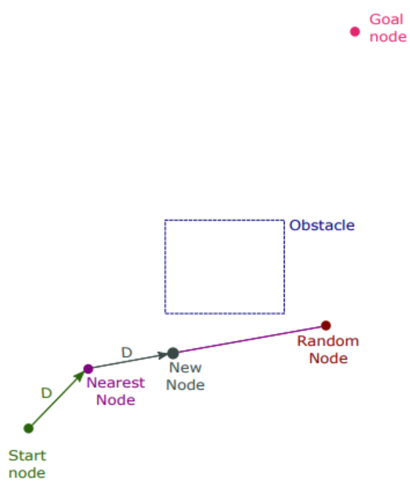

Figure 7. RRT Second Iterate.

It has been shown that there is unity probability to find a path to the goal using RRT, given there is no limit on the number of vertices or time. The algorithm can thus be said to have probabilistic completeness. Even if a path always can be found using RRT in most cases it does not represent an optimal path since it usually contains many sharp turns, even in obstacle free areas. It was later shown that the probability for RRT converging to the optimal path is in fact equal to zero [7].

E. Cost Calculations

The cost function for the RRT-u method is based on calculating the time it takes for the vehicle to travel between two nodes, so that when looking at a planning tree, the nearest neighbor is then simply defined as the node from which it takes the shortest time to travel. It does not necessarily have to be the node closest in space. The exact implementation of what counts as a cost is actually quite arbitrary and can, if desired, be changed to something else, such as for instance energy consumption. The algorithm is defined so that the accelerations (ax, ay, az) are the free parameters and the velocities ($vx1$, $vy1$, $vz1$) are functions of the accelerations. There are a few limiting conditions which are given by the dynamical model used by this method, which must be considered when doing the cost minimization. To start with, the vehicle is approximated with a point mass which can be freely moved in any direction of three-dimensional space. One of the limiting conditions is that the acceleration in every dimension is constant throughout the entire trajectory between two nodes. There is also an upper limit for the absolute value of the acceleration. Then, finally, the absolute value of the velocity for each dimension is always required to be below a maximum value. Analyzing the problem further, given the above limitations of the model, there will only be a limited number of cases to consider. Either the acceleration for some dimension is at its maximum value during the whole course of the trajectory, or the velocity of the same dimension reaches its maximum value at the end of the trajectory, while the acceleration is at a constant value lower than the maximum. There are three dimensions to investigate and consequently three distances that needs to be traversed when travelling from one node to another, which are equal to the differences between the end positions and the start positions

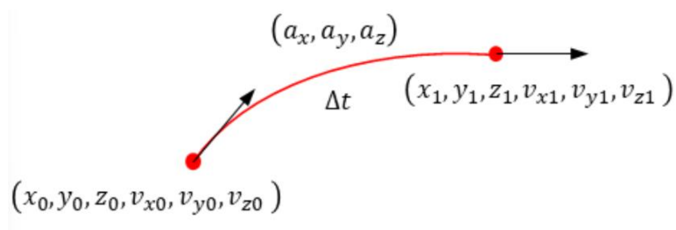

Figure 8. The quantities involved in the local motion planning between two vertices

III. Simulation and Result

The simulation model of PX4 UAV was loaded into the ROS system under Ubuntu environment, a simple UAV flight environment scenario was set up by Gazebo11. The RRT path-planning program is loaded into the simulated UAV and simulated flight tests are performed. The main components of the simulation system are have to be able to communicate one and another, and the simulation going well as shows in figure 9. Terminal command is used to control those separate parts so that the simulation can be completely running. Through mavros node which is connected with FCU model, ROS will publish processed data (position, velocity) from RRT topic in Gazebo simulator.

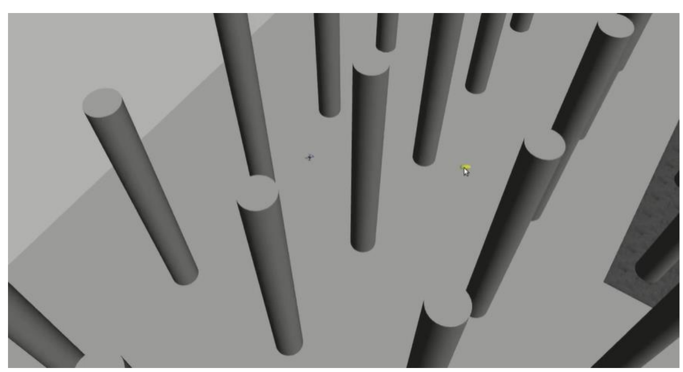

Figure 9. Simulation on gazebo

Upon deriving a viable path (depicted in green) via the RRT technique, this study engaged in optimization to streamline it, thus minimizing the number of waypoints (depicted in blue). The initial path, constituting a sequence of interconnected nodes within the RRT tree, underwent refinement by randomly selecting two nodes from the path and endeavoring to establish a direct connection between them, thereby circumventing intermediate nodes.

The outcome illustrates the optimized path, which was subjected to testing within a more intricate obstacle-laden environment. Initially, the viable path encompassed 102 waypoints. However, through optimization, this figure was substantially reduced to a mere 8 waypoints.

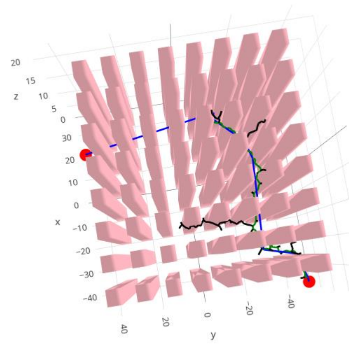

Figure 10. Trajectory stereogram chart result

Table 2 shows the eigenvalues of the path-cost statistics and time statistics of the four algorithms in obtaining the initial solution, where c_{mean}, c_{median}, c_{max}, and c_{min} are the mean, median, maximum, and minimum values of the 1000 sets of path cost, respectively. Meanwhile, t_{mean}, t_{median}, t_{max}, and t_{min} are the mean, median, maximum, and minimum values of 1000 sets of search time statistics, respectively.

TABLE II. THE RESULT OF PATH COST AND TIME STATISTIC

c_{mean}/m	56, 83
c_{median}/m	56,51
c_{max}/m	77,67
c_{min}/m	44,26
t_{mean}/m	0,90
t_{median}/m	0,26
t_{max}/m	21,74
t_{min}/m	0,02

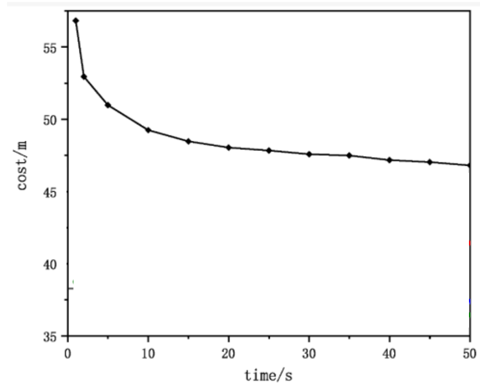

Figure 11. Optimal convergence diagram

IV. CONCLUSION

The simulation system, combining Gazebo, PX4 SITL, ROS, and the RRT path planning algorithm, offers a comprehensive framework for UAV simulation and path planning. Through effective communication between components and optimization techniques, it facilitates efficient path generation for UAV navigation in complex environments.

The simulation system proposed in this study is based on the Robot Operating System (ROS) framework and consists of three main components: the quadrotor model in Gazebo, PX4's Software In The Loop (SITL), and the Rapidly-Exploring Random Tree (RRT) path planning algorithm. The quadrotor model in Gazebo, representing a UAV, operates within a six-degree-of-freedom framework and is controlled by four inputs: thrust force along the Z-axis and moments for roll, pitch, and yaw. This model is simulated using PX4's SITL, allowing for command input from a Linux terminal.

ROS facilitates communication between the components, enabling data exchange between Gazebo, SITL, and the RRT algorithm. MAVROS, a ROS package, facilitates interaction with the autopilot software using the MAVLink protocol. RQt provides a user-friendly interface for interacting with the system, while RViz offers 3D visualization capabilities, crucial for visualizing the UAV's path iterations.The RRT algorithm is employed for path planning, constructing a tree data structure to navigate an obstacle-laden environment. It rapidly expands the path from the starting point to the goal, leveraging randomness to avoid local optimization traps. Despite its probabilistic completeness, RRT does not always yield an optimal path due to its inherent randomness.

In the simulation results, the initial path generated by RRT is optimized to minimize the number of waypoints, resulting in a more streamlined trajectory. The optimization process substantially reduces the waypoint count, enhancing the efficiency of the path while maintaining feasibility. Statistical analysis of path cost and time demonstrates the effectiveness of the optimized path.

ACKNOWLEDGMENT

I firmly say thanks to Mr. Yayan Prima Nugraha and Mr Rizki Putra Prastio as partner research for implementing RRT algorithm. I also say thanks to Muhammad Irsyad Asyarie, Jasmin Adlina Hapsari, Tiara Tatyanadewi Benjamin,

Muhummad Wisnu Imam Satrio and Robby Setyanto as partner internship project from PT Irostech Solusi Inteligen.

REFERENCES

[1] Chamola, V.; Kotesh, P.; Agarwal, A.; Gupta, N.; Guizani, M. A comprehensive review of unmanned aerial vehicle attacks and neutralization techniques. Ad Hoc Netw. 2021, 111, 102324. [Google Scholar] [CrossRef] [PubMed]

[2] Ha, S.W.; Paul, Q.; Moon, Y.H. Improvement of UAV attitude information estimation performance using image processing and kalman filter. J. Converg. Inf. Technol. 2018, 8, 135–142. [Google Scholar]

[3] Evers, L.; Dollevoet, T.; Barros, A.I.; Monsuur, H. Robust UAV mission planning. Ann. Oper. Res. 2014, 222, 293–315. [Google Scholar] [CrossRef]

[4] Wang, Z.; Yue, P. Marine Island UAV Aerial Photography: A Path-Planning Algorithm-Based Study. J. Coast. Res. 2020, 106, 642–645. [Google Scholar]

[5] Wang, S.; Xu, S.; Yu, C.; Wu, H.; Liu, Q.; Liu, D.; Jin, L.; Zheng, Y.; Song, J.; He, X. Obstacle Avoidance and Profile Ground Flight Test and Analysis for Plant Protection UAV. Drones 2022, 6, 125. [Google Scholar] [CrossRef]

[6] Li, J.; Liu, H.; Lai, K.K.; Ram, B. Vehicle and UAV Collaborative Delivery Path Optimization Model. Mathematics 2022, 10, 3744. [Google Scholar] [CrossRef]

[7] Papadopoulou, E.E.; Vasilakos, C.; Zouros, N.; Soulakellis, N. DEM-Based UAV Flight Planning for 3D Mapping of Geosites: The Case of Olympus Tectonic Window, Lesvos, Greece. ISPRS Int. J. Geo-Inf. 2021, 10, 535. [Google Scholar] [CrossRef]

[8] Sun, W.; Fan, K.; Huang, T.; Cui, J. Path Optimization of UAV Inspection Suspended Track Based on Dynamic Adaptive Ant Colony Optimization. In Proceedings of the 2022 6th International Conference on Automation, Control and Robots (ICACR), Shanghai, China, 23–25 September 2022; pp. 142–147. [Google Scholar]

[9] Samar, R.; Rehman, A. Autonomous terrain-following for unmanned air vehicles. Mechatronics 2011, 21, 844–860. [Google Scholar] [CrossRef]

[10] Stentz, A. Optimal and efficient path planning for partially known environments. In Intelligent Unmanned Ground Vehicles: Autonomous Navigation Research at Carnegie Mellon; Springer: Berlin/Heidelberg, Germany, 1997; pp. 203–220. [Google Scholar]

[11] Daniel, K.; Nash, A.; Koenig, S.; Felner, A. Theta*: Any-angle path planning on grids. J. Artif. Intell. Res. 2010, 39, 533–579. [Google Scholar] [CrossRef]

[12] Ferguson, D.; Stentz, A. Field D*: An interpolation-based path planner and replanner. In Robotics Research: Results of the 12th International Symposium ISRR; Springer: Berlin/Heidelberg, Germany, 2007; pp. 239–253. [Google Scholar]

[13] Tu, J.; Yang, S.X. Genetic algorithm based path planning for a mobile robot. In Proceedings of the 2003 IEEE International Conference on Robotics and Automation (Cat. No. 03CH37422), Taipei, Taiwan, 14–19 September 2003; Volume 1, pp. 1221–1226. [Google Scholar]

[14] Dorigo, M.; Maniezzo, V.; Colorni, A. Ant system: Optimization by a colony of cooperating agents. IEEE Trans. Syst. Man Cybern. Part B Cybern. 1996, 26, 29–41. [Google Scholar] [CrossRef] [PubMed]

[15] Oleiwi, B.K.; Roth, H.; Kazem, B.I. A hybrid approach based on ACO and GA for multi objective mobile robot path planning. Appl. Mech. Mater. 2014, 527, 203–212. [Google Scholar] [CrossRef]

[16] Kennedy, J.; Eberhart, R.C. A discrete binary version of the particle swarm algorithm. In Proceedings of the 1997 IEEE International Conference on Systems, Man, and Cybernetics, Computational Cybernetics and Simulation, Orlando, FL, USA, 12–15 October 1997; Volume 5, pp. 4104–4108. [Google Scholar]

[17] Khuswendi, T.; Hindersah, H.; Adiprawita, W. Uav path planning using potential field and modified receding horizon A* 3D algorithm. In Proceedings of the 2011 International Conference on Electrical Engineering and Informatics, Bandung, Indonesia, 17–19 July 2011; pp. 1–6. [Google Scholar]

[18] Saravanakumar, S.; Asokan, T. Multipoint potential field method for path planning of autonomous underwater vehicles in 3D space. Intell. Serv. Robot. 2013, 6, 211–224. [Google Scholar] [CrossRef]

[19] Raja, R.; Dutta, A.; Venkatesh, K.S. New potential field method for rough terrain path planning using genetic algorithm for a 6-wheel rover. Robot. Auton. Syst. 2015, 72, 295–306. [Google Scholar] [CrossRef]

[20] LaValle, S.M. Rapidly-exploring random trees: A new tool for path planning. In The Annual Research Report; Iowa State University: Ames, IA, USA, 1998. [Google Scholar]

[21] Kavraki, L.E.; Kolountzakis, M.N.; Latombe, J.C. Analysis of probabilistic roadmaps for path planning. IEEE Trans. Robot. Autom. 1998, 14, 166–171. [Google Scholar] [CrossRef]

[22] Kuffner, J. RRT-Connect: An efficient approach to single-query path planning. In Proceedings of the IEEE International Conference on Robotics and Automation, San Francisco, CA, USA, 24–28 April 2000; pp. 473–479. [Google Scholar]

[23] Karaman, S.; Walter, M.R.; Perez, A.; Frazzoli, E.; Teller, S. Anytime motion planning using the RRT. In Proceedings of the 2011 IEEE International Conference on Robotics and Automation, Shanghai, China, 9–13 May 2011; pp. 1478–1483. [Google Scholar]

[24] Jeong, I.B.; Lee, S.J.; Kim, J.H. Quick-RRT*: Triangular inequality-based implementation of RRT* with improved initial solution and convergence rate. Expert Syst. Appl. 2019, 123, 82–90. [Google Scholar] [CrossRef]

[25] Urmson, C.; Simmons, R. Approaches for heuristically biasing RRT growth. In Proceedings of the 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003) (Cat. No. 03CH37453), Las Vegas, NV, USA, 27–31 October 2003; Volume 2, pp. 1178–1183. [Google Scholar]

[26] Gammell, J.D.; Srinivasa, S.S.; Barfoot, T.D. Informed RRT: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic. In Proceedings of the 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems, Chicago, IL, USA, 14–18 September 2014; pp. 2997–3004. [Google Scholar]

[27] Li, Y.; Wei, W.; Gao, Y.; Wang, D.; Fan, Z. PQ-RRT*: An improved path planning algorithm for mobile robots. Expert Syst. Appl. 2020, 152, 113425. [Google Scholar] [CrossRef]

[28] Liao, B.; Wan, F.; Hua, Y.; Ma, R.; Zhu, S.; Qing, X. F-RRT*: An improved path planning algorithm with improved initial solution and convergence rate. Expert Syst. Appl. 2021, 184, 115457. [Google Scholar] [CrossRef]

[29] Guo, Y.; Liu, X.; Jiang, W.; Zhang, W. Collision-Free 4D Dynamic Path Planning for Multiple UAVs Based on Dynamic Priority RRT* and Artificial Potential Field. Drones 2023, 7, 180. [Google Scholar] [CrossRef]